

```

In [1]: import pennylane as qml
from pennylane.optimize import NesterovMomentumOptimizer
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, accuracy_score, precision_score

# Load dataset
data = load_breast_cancer()
X, y = data.data, data.target

# Preprocess data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,

# Reduce to 4 features for quantum circuit feasibility
X_train = X_train[:, :4]
X_test = X_test[:, :4]

# Quantum device
n_qubits = 4
dev = qml.device("default.qubit", wires=n_qubits)

# Variational circuit
def variational_circuit(params, x):
    for i in range(n_qubits):
        qml.RY(x[i], wires=i)
    qml.templates.StronglyEntanglingLayers(params, wires=range(n_qubits))
    return qml.expval(qml.PauliZ(0))

@qml.qnode(dev)
def circuit(params, x):
    return variational_circuit(params, x)

# Prediction function
def predict(params, X):
    preds = []
    for x in X:
        output = circuit(params, x)
        preds.append(1 if output >= 0 else 0)
    return np.array(preds)

# Cost function
def cost(params, X, y):
    preds = np.array([circuit(params, x) for x in X])
    return np.mean((preds - (2*y - 1))**2) # MSE with {+1, -1}

# Training
np.random.seed(42)
params = 0.01 * np.random.randn(3, n_qubits, 3) # Shape depends on Layers and q
opt = NesterovMomentumOptimizer(stepsize=0.5)

epochs = 30
for epoch in range(epochs):
    params = opt.step(lambda p: cost(p, X_train, y_train), params)
    if epoch % 5 == 0:

```

```

        loss = cost(params, X_train, y_train)
        print(f"Epoch {epoch}: Loss = {loss:.4f}")

# Evaluation
y_pred = predict(params, X_test)

# Metrics
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")

```

c:\Users\Mr. Nitin\Desktop\quantum_ML\lib\site-packages\pennylane_grad.py:216: UserWarning: Attempted to differentiate a function with no trainable parameters. If this is unintended, please add trainable parameters via the 'requires_grad' attribute or 'argnum' keyword.

warnings.warn(

Epoch 0: Loss = 0.9647
 Epoch 5: Loss = 0.9647
 Epoch 10: Loss = 0.9647
 Epoch 15: Loss = 0.9647
 Epoch 20: Loss = 0.9647
 Epoch 25: Loss = 0.9647

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.26	0.41	43
1	0.69	1.00	0.82	71
accuracy			0.72	114
macro avg	0.84	0.63	0.61	114
weighted avg	0.81	0.72	0.66	114

Accuracy: 0.7193
 Precision: 0.6893
 Recall: 1.0000
 F1 Score: 0.8161